

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“Insertion Sort,Bubble Sort, dan Selection Sort”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data Pekan 9 ini dapat diselesaikan dengan baik. Praktikum pada pekan ini membahas tentang **Binary Tree (B-Tree)** sebagai salah satu struktur data non-linear yang digunakan untuk menyimpan data dalam bentuk hierarki. Struktur data ini memungkinkan proses penyimpanan, pencarian, dan pengelolaan data dilakukan dengan lebih terorganisir dibandingkan struktur data linear.

Dalam praktikum ini, mahasiswa mempelajari konsep dasar Binary Tree, mulai dari pembuatan simpul (node), pembentukan pohon biner, hingga berbagai operasi yang dapat dilakukan seperti menambahkan simpul, menghitung jumlah simpul, dan melakukan penelusuran pada pohon. Implementasi dilakukan menggunakan bahasa pemrograman Java untuk membantu memahami penerapan konsep Binary Tree secara nyata.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa mendatang. Semoga laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai struktur data Binary Tree.

Padang, 10 Juni 2026

Aliyatar Rafi Ahmad

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengelola dan menyimpan data agar dapat diakses serta diproses secara efisien. Pemilihan struktur data yang tepat dapat meningkatkan kinerja program dan mempermudah penyelesaian berbagai permasalahan komputasi. Oleh karena itu, pemahaman mengenai berbagai jenis struktur data menjadi hal yang penting bagi mahasiswa dalam mempelajari pemrograman.

Pada praktikum pekan ini dipelajari dua struktur data non-linear, yaitu **Binary Tree** dan **Graph**. Binary Tree merupakan struktur data berbentuk pohon yang setiap simpulnya memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Struktur ini banyak digunakan dalam proses pencarian, pengurutan data, dan representasi hubungan hierarkis. Selain itu, dipelajari juga Graph yang digunakan untuk merepresentasikan hubungan antar objek dalam bentuk simpul dan sisi.

Untuk melakukan penelusuran pada struktur Binary Tree dan Graph, digunakan beberapa metode traversal. Pada Binary Tree digunakan traversal **Inorder, Preorder, dan Postorder**, sedangkan pada Graph digunakan algoritma **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. Melalui praktikum ini mahasiswa diharapkan dapat memahami konsep dasar Binary Tree dan Graph serta mampu mengimplementasikan berbagai metode traversal menggunakan bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar Binary Tree sebagai struktur data non-linear.
2. Mengimplementasikan operasi dasar pada Binary Tree seperti pencarian data dan perhitungan jumlah simpul.
3. Mengimplementasikan metode traversal Binary Tree yaitu Inorder, Preorder, dan Postorder.

4. Mengimplementasikan algoritma penelusuran Graph yaitu Depth First Search (DFS) dan Breadth First Search (BFS).

1.3 Manfaat

1. Menambah pemahaman mengenai struktur data non-linear, khususnya Binary Tree dan Graph.
2. Melatih kemampuan dalam mengimplementasikan struktur data menggunakan bahasa pemrograman Java.
3. Membantu memahami cara kerja algoritma traversal pada Binary Tree dan Graph.

BAB II

PEMBAHASAN

2.1 BinaryTree

Binary Tree merupakan struktur data non-linear yang terdiri dari kumpulan simpul (*node*) yang saling terhubung dalam bentuk hierarki. Setiap simpul pada Binary Tree dapat memiliki paling banyak dua anak, yaitu anak kiri (*left child*) dan anak kanan (*right child*). Simpul paling atas disebut *root*, sedangkan simpul yang tidak memiliki anak disebut *leaf node*.

Binary Tree banyak digunakan dalam berbagai aplikasi komputer, seperti pencarian data, pengurutan data, dan representasi struktur hierarki. Keunggulan Binary Tree adalah kemampuannya dalam mengorganisasi data secara terstruktur sehingga proses pencarian dan pengolahan data dapat dilakukan lebih efisien.

2.1.1 Traversal

Pada Binary Tree terdapat proses yang disebut traversal, yaitu proses mengunjungi setiap simpul dalam tree dengan urutan tertentu. Terdapat tiga metode traversal yang umum digunakan, yaitu **Inorder**, **Preorder**, dan **Postorder**. Traversal Inorder dilakukan dengan urutan *Left → Root → Right*, traversal Preorder dilakukan dengan urutan *Root → Left → Right*, sedangkan traversal Postorder dilakukan dengan urutan *Left → Right → Root*. Ketiga metode tersebut digunakan untuk menampilkan atau memproses data yang tersimpan pada Binary Tree sesuai kebutuhan.

2.2 Graph

Graph merupakan struktur data yang terdiri dari sekumpulan simpul (*vertex*) dan hubungan antar simpul (*edge*). Berbeda dengan tree yang memiliki struktur hierarki, graph memungkinkan setiap simpul terhubung dengan banyak simpul lainnya. Struktur data ini banyak digunakan untuk

merepresentasikan jaringan komputer, peta, media sosial, serta berbagai hubungan antar objek.

2.3 Depth First Search (DFS) dan Breadth First Search (BFS)

Depth First Search (DFS) dan Breadth First Search (BFS) merupakan dua algoritma yang digunakan untuk melakukan penelusuran pada graph. DFS bekerja dengan menelusuri satu cabang hingga mencapai simpul terdalam sebelum kembali ke simpul sebelumnya (*backtracking*). Algoritma ini biasanya diimplementasikan menggunakan rekursi atau struktur data stack.

Sementara itu, BFS melakukan penelusuran berdasarkan tingkat kedekatan dari simpul awal. BFS menggunakan struktur data queue sehingga simpul yang pertama masuk akan diproses terlebih dahulu. DFS cocok digunakan untuk menjelajahi seluruh jalur yang mungkin pada graph, sedangkan BFS sering digunakan untuk mencari jalur terpendek pada graph yang tidak memiliki bobot.

2.4 Implementasi pada Java

Pada praktikum pekan ke-9 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **Binary Tree** serta algoritma **Graph Traversal** menggunakan metode **Depth First Search (DFS)** dan **Breadth First Search (BFS)** dalam bahasa pemrograman Java. Berikut kode program yang telah dibuat:

2.4.1 Node_2511533031

```
1. package pekan9_2511533031;
2.
3. public class Node_2511533031 {
4.     int data_3031;
5.     Node_2511533031 left_3031;
6.     Node_2511533031 right_3031;
7.     public Node_2511533031(int data_3031) {
8.         this.data_3031 = data_3031;
9.         left_3031 = null;
10.        right_3031 = null;
11.    }
12.    public void setLeft_3031(Node_2511533031 node_3031) {
13.        if (left_3031 == null)
14.            left_3031 = node_3031;
```

```

15. }
16. public void setRight_3031(Node_2511533031 node_3031) {
17.     if (right_3031 == null)
18.         right_3031 = node_3031;
19. }
20. public Node_2511533031 getLeft_3031() {
21.     return left_3031;
22. }
23. public Node_2511533031 getRight_3031() {
24.     return right_3031;
25. }
26. public int getData_3031() {
27.     return data_3031;
28. }
29. public void setData_3031() {
30.     this.data_3031 = data_3031;
31. }
32. void printPreorder_3031(Node_2511533031 node_3031) {
33.     if (node_3031 == null)
34.         return;
35.     System.out.println(node_3031.data_3031 + " ");
36.     printPreorder_3031(node_3031.left_3031);
37.     printPreorder_3031(node_3031.right_3031);
38. }
39. void printPostorder_3031(Node_2511533031 node_3031) {
40.     if (node_3031 == null)
41.         return;
42.     printPostorder_3031(node_3031.left_3031);
43.     printPostorder_3031(node_3031.right_3031);
44.     System.out.print(node_3031.data_3031 + " ");
45. }
46. void printInorder_3031(Node_2511533031 node_3031) {
47.     if (node_3031 == null)
48.         return;
49.     printInorder_3031(node_3031.left_3031);
50.     System.out.print(node_3031.data_3031 + " ");
51.     printInorder_3031(node_3031.right_3031);
52. }
53. public String print_3031 () {
54.     return this.print_3031 ("",true,"");
55. }
56. public String print_3031 (String prefix_3031, boolean
isTail_3031, String sb_3031) {
57.     if (right_3031 != null) {
58.         right_3031.print_3031 (prefix_3031 +
(isTail_3031 ? "|" : " "), false, sb_3031 );
59.     }
60.     System.out.println(prefix_3031 + (isTail_3031 ? "\\-
-" : "/--")+ data_3031);
61.     if (left_3031 != null) {
62.         left_3031.print_3031(prefix_3031 +
(isTail_3031 ? " " : " | "), true, sb_3031);
63.     }
64.     return sb_3031;

```

```
65. }  
66. }
```

Program `Node_2511533031` merupakan kelas yang digunakan untuk merepresentasikan simpul (*node*) pada struktur data Binary Tree. Setiap node memiliki data yang disimpan pada atribut `data_3031` serta dua referensi, yaitu `left_3031` dan `right_3031`, yang menunjuk ke anak kiri dan anak kanan.

Kelas ini menyediakan method untuk menambahkan anak kiri dan kanan (`setLeft_3031()` dan `setRight_3031()`), mengambil data node (`getData_3031()`), serta melakukan traversal tree menggunakan metode **Preorder**, **Inorder**, dan **Postorder**. Selain itu, terdapat method `print_3031()` yang digunakan untuk menampilkan struktur Binary Tree dalam bentuk pohon sehingga hubungan antar node dapat terlihat dengan lebih jelas.

2.4.2 BTree_2511533031

```
1. package pekan9_2511533031;  
2.  
3. public class BTree_2511533031 {  
4.     private Node_2511533031 root_3031;  
5.     private Node_2511533031 currentNode_3031;  
6.     public BTree_2511533031() {  
7.         root_3031 = null;  
8.     }  
9.     public boolean search_3031(int data_3031) {  
10.         return search_3031(root_3031, data_3031);  
11.     }  
12.     private boolean search_3031(Node_2511533031 node_3031,  
13. int data_3031) {  
14.         if (node_3031.getData_3031() == data_3031)  
15.             return true;  
16.         if (node_3031.getLeft_3031() != null)  
17.             if (search_3031(node_3031.getLeft_3031(),  
18. data_3031))  
19.                 return true;  
20.         if (node_3031.getRight_3031() != null)  
21.             if (search_3031(node_3031.getRight_3031(),  
22. data_3031))  
23.                 return true;  
24.         return false;  
25.     }  
26.     public void printInorder_3031() {
```



```

24.         root_3031.printInorder_3031(root_3031);
25.     }
26.     public void printPreOrder_3031() {
27.         root_3031.printPostorder_3031(root_3031);
28.     }
29.     public void printPostOrder_3031() {
30.         root_3031.printPostorder_3031(root_3031);
31.     }
32.     public Node_2511533031 getRoot_3031() {
33.         return root_3031;
34.     }
35.     public boolean isEmpty_3031 () {
36.         return root_3031 == null;
37.     }
38.     public int countNodes_3031 () {
39.         return countNodes_3031 (root_3031);
40.     }
41.     private int countNodes_3031 (Node_2511533031 node_3031) {
42.         int count_3031 = 1;
43.         if (node_3031 == null) {
44.             return 0;
45.         } else {
46.             count_3031 += countNodes_3031
47.                 (node_3031.getLeft_3031());
48.             count_3031 += countNodes_3031
49.                 (node_3031.getRight_3031());
50.             return count_3031;
51.         }
52.     }
53.     public void print_3031 () {
54.         root_3031.print_3031();
55.     }
56.     public Node_2511533031 getCurrent_3031 () {
57.         return currentNode_3031;
58.     }
59.     public void setCurrent_3031 (Node_2511533031 node_3031) {
60.         this.currentNode_3031 = node_3031;
61.     }
62.     public void setRoot_3031 (Node_2511533031 root_3031) {
63.         this.root_3031 = root_3031;
64.     }

```

Program BTree_2511533031 merupakan kelas yang digunakan untuk mengelola struktur data Binary Tree. Kelas ini memiliki atribut root_3031 sebagai akar pohon dan currentNode_3031 sebagai penunjuk node yang sedang aktif. Konstruktor BTree_2511533031() digunakan untuk membuat tree kosong dengan nilai root null.

Kelas ini menyediakan beberapa operasi dasar Binary Tree, seperti `search_3031()` untuk mencari data pada tree secara rekursif, `countNodes_3031()` untuk menghitung jumlah node yang terdapat dalam tree, serta `isEmpty_3031()` untuk memeriksa apakah tree masih kosong atau tidak. Selain itu, terdapat method `printInorder_3031()`, `printPreOrder_3031()`, dan `printPostOrder_3031()` yang digunakan untuk melakukan traversal pada Binary Tree.

Program juga menyediakan method `setRoot_3031()`, `getRoot_3031()`, `setCurrent_3031()`, dan `getCurrent_3031()` untuk mengatur maupun mengambil node tertentu. Method `print_3031()` digunakan untuk menampilkan struktur Binary Tree dalam bentuk pohon sehingga susunan node dapat dilihat dengan lebih mudah.

2.4.3 BTreeDriver_2511533031

```
1. package pekan9_2511533031;
2.
3. public class BtreeDriver_2511533031 {
4.     public static void main(String[] args) {
5.         // membuat pohon
6.         BTree_2511533031 tree_3031 = new BTree_2511533031();
7.         System.out.print("Jumlah Simpul awal pohon: ");
8.         System.out.println(tree_3031.countNodes_3031());
9.         // menambahkan simpul data 1
10.        Node_2511533031 root_3031 = new Node_2511533031(1);
11.        // menjadikan simpul 1 sebagai root
12.        tree_3031.setRoot_3031(root_3031);
13.        System.out.println("Jumlah simpul jika hanya ada root");
14.        System.out.println(tree_3031.countNodes_3031());
15.        Node_2511533031 node2_3031 = new Node_2511533031(2);
16.        Node_2511533031 node3_3031 = new Node_2511533031(3);
17.        Node_2511533031 node4_3031 = new Node_2511533031(4);
18.        Node_2511533031 node5_3031 = new Node_2511533031(5);
19.        Node_2511533031 node6_3031 = new Node_2511533031(6);
20.        Node_2511533031 node7_3031 = new Node_2511533031(7);
21.        Node_2511533031 node8_3031 = new Node_2511533031(8);
22.        Node_2511533031 node9_3031 = new Node_2511533031(9);
23.        root_3031.setLeft_3031(node2_3031);
24.        node2_3031.setLeft_3031(node4_3031);
25.        node2_3031.setRight_3031(node5_3031);
26.        node4_3031.setRight_3031(node8_3031);
27.        root_3031.setRight_3031(node3_3031);
28.        node3_3031.setLeft_3031(node6_3031);
29.        node3_3031.setRight_3031(node7_3031);
30.        node6_3031.setLeft_3031(node9_3031);
```

```

31. // set root_3031
32. tree_3031.setCurrent_3031(tree_3031.getRoot_3031());
33. System.out.println("menampilkan simpul terakhir:");
34. System.out.println(tree_3031.getCurrent_3031().getData_3031
    ());
35. System.out.println("Jumlah simpul; setelah simpul 7
    ditambahkan");
36. System.out.println(tree_3031.countNodes_3031());
37. System.out.println("InOrder: ");
38. tree_3031.printInorder_3031();
39. System.out.println("\nPreorder: ");
40. tree_3031.printInorder_3031();
41. System.out.println("\nPostorder: ");
42. tree_3031.printPostOrder_3031();
43. System.out.println("\nMenampilkan simpul dalam bentuk
    pohon");
44. tree_3031.print_3031();
45. }
46. }

```

Program BtreeDriver_2511533031 berfungsi sebagai kelas utama (*driver class*) untuk menguji implementasi Binary Tree yang telah dibuat. Pada program ini dibuat sebuah objek BTree_2511533031, kemudian beberapa node dengan nilai 1 sampai 9 dibuat dan dihubungkan sehingga membentuk struktur Binary Tree.

Setelah tree terbentuk, program menampilkan jumlah node yang ada pada tree menggunakan method `countNodes_3031()`, serta mengatur root sebagai node aktif dengan method `setCurrent_3031()`. Program juga menampilkan nilai node aktif menggunakan method `getCurrent_3031()`.

Selain itu, program melakukan traversal Binary Tree menggunakan metode **Inorder**, **Preorder**, dan **Postorder** untuk menampilkan urutan kunjungan node. Pada bagian akhir, method `print_3031()` digunakan untuk menampilkan struktur Binary Tree dalam bentuk pohon sehingga hubungan antar node dapat dilihat dengan lebih jelas.

2.4.4 GraphTraversal_2511533031

```
1. package pekan9_2511533031;
2. import java.util.*;
3. public class GraphTraversal_2511533031 {
4.     private Map<String, List<String>> graph_3031 = new
        HashMap<>();
5.
6.     //Menambahkan edge (graf tak berarah)
7.     public void addEdge(String node1, String node2) {
8.         graph_3031.putIfAbsent(node1, new ArrayList<>());
9.         graph_3031.putIfAbsent(node2, new ArrayList<>());
10.        graph_3031.get(node1).add(node2);
11.        graph_3031.get(node2).add(node1);
12.    }
13.    // Menampilkan graf awal
14.    public void printGraph_3031() {
15.        System.out.println("Graf Awal (adjacency List):");
16.        for (String node_3031 : graph_3031.keySet()) {
17.            System.out.print(node_3031 + " -> ");
18.            List<String> neighbors_3031 =
                graph_3031.get(node_3031);
19.            System.out.println(String.join(", ",
                neighbors_3031));
20.        }
21.        System.out.println();
22.    }
23.    // DFS rekursi
24.    public void dfs_3031(String start_3031) {
25.        Set<String> visited_3031 = new HashSet<>();
26.        System.out.println("Penelusuran DFS:");
27.        dfsHelper_3031(start_3031, visited_3031);
28.        System.out.println();
29.    }
30.    private void dfsHelper_3031(String current_3031,
        Set<String> visited) {
31.        if (visited.contains(current_3031)) return;
32.        visited.add(current_3031);
33.        System.out.print(current_3031 + " ");
34.        for (String neighbor_3031 :
            graph_3031.getOrDefault(current_3031, new ArrayList<>())) {
35.            dfsHelper_3031(neighbor_3031, visited);
36.        }
37.    }
38.    // BFS Iteratif
39.    public void bfs_3031(String start) {
40.        Set<String> visited_3031 = new HashSet<>();
41.        Queue<String> queue_3031 = new LinkedList<>();
42.
43.        queue_3031.add(start);
44.        visited_3031.add(start);
45.        System.out.println("Penelusuran BFS:");
46.        while (!queue_3031.isEmpty()) {
47.            String current_3031 = queue_3031.poll();
```

```

48.         System.out.print(current_3031 + " ");
49.         for (String neighbor_3031 :
graph_3031.getDefault(current_3031, new ArrayList<>())) {
50.             if
51.             (!visited_3031.contains(neighbor_3031)) {
52.                 queue_3031.add(neighbor_3031);
53.                 visited_3031.add(neighbor_3031);
54.             }
55.         }
56.         System.out.println();
57.     }
58.     // Main
59.     public static void main(String[] args) {
60.         GraphTraversal_2511533031 graph_3031 = new
GraphTraversal_2511533031();
61.         // Contoh graf: A-B, A-C, B-D, B-E
62.         graph_3031.addEdge("A", "B");
63.         graph_3031.addEdge("A", "C");
64.         graph_3031.addEdge("B", "D");
65.         graph_3031.addEdge("B", "E");
66.         // Cetak graf awal
67.         System.out.println("Graf Awal adalah: ");
68.         graph_3031.printGraph_3031();
69.         // Lakukan penelusuran
70.         graph_3031.dfs_3031("A");
71.         graph_3031.bfs_3031("A");
72.     }
73. }

```

Program GraphTraversal_2511533031 digunakan untuk mengimplementasikan struktur data **Graph** serta melakukan proses penelusuran menggunakan algoritma **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. Graph disimpan menggunakan struktur **Adjacency List** dengan memanfaatkan **HashMap** dan **ArrayList** sehingga hubungan antar simpul dapat direpresentasikan dengan lebih efisien.

Method `addEdge()` digunakan untuk menambahkan hubungan (*edge*) antara dua simpul pada graph, sedangkan method `printGraph_3031()` digunakan untuk menampilkan bentuk graph dalam format adjacency list. Program ini juga menyediakan method `dfs_3031()` untuk melakukan penelusuran secara mendalam menggunakan rekursi, serta method `bfs_3031()` untuk melakukan penelusuran per tingkat menggunakan struktur data queue.

Pada method `main()`, dibuat sebuah graph sederhana yang terdiri dari simpul A, B, C, D, dan E. Setelah graph ditampilkan, program menjalankan proses traversal menggunakan metode DFS dan BFS yang dimulai dari simpul A, sehingga pengguna dapat melihat perbedaan urutan penelusuran dari kedua algoritma tersebut.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data **Binary Tree** digunakan untuk menyimpan data dalam bentuk hierarki yang terdiri dari root, node, dan child node. Melalui implementasi Binary Tree, dapat dilakukan berbagai operasi seperti penambahan node, pencarian data, perhitungan jumlah node, serta traversal menggunakan metode **Inorder**, **Preorder**, dan **Postorder**.

Selain itu, praktikum juga membahas struktur data **Graph** beserta algoritma penelusurannya, yaitu **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. DFS melakukan penelusuran hingga ke simpul terdalam sebelum kembali ke simpul sebelumnya, sedangkan BFS menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal. Implementasi kedua algoritma tersebut membantu dalam memahami cara penelusuran data pada graph secara efektif.

Melalui praktikum ini, mahasiswa dapat memahami konsep struktur data non-linear serta penerapannya dalam bahasa pemrograman Java, sehingga dapat menjadi dasar dalam pengembangan program yang lebih kompleks pada bidang ilmu komputer.

DAFTAR PUSTAKA

- [1] Programiz, "Binary Tree Data Structure," Programiz, [Online]. Available:
https://www.programiz.com/dsa/binary-tree?utm_source=chatgpt.com.
[Accessed 10 June 2026].

- [2] Programiz, "Graph Data Structure," Programiz, [Online]. Available:
https://www.programiz.com/dsa/graph?utm_source=chatgpt.com. [Accessed
10 June 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-9:

```
1. package pekan9_2511533031;
2.
3. import javax.swing.*;
4. import java.awt.*;
5. import java.util.*;
6.
7. public class PetaKampus_2511533031 extends JFrame {
8.
9.     private Map<String, java.util.List<String>> graph_3031;
10.
11.     private JComboBox<String> cbAwal_3031;
12.     private JComboBox<String> cbTujuan_3031;
13.
14.     private JTextArea graphArea_3031;
15.     private JTextArea hasilArea_3031;
16.
17.     private Set<String> visited_3031 = new LinkedHashSet<>();
18.     private java.util.List<String> path_3031 = new ArrayList<>();
19.
20.     public PetaKampus_2511533031() {
21.         initGraph_3031();
22.         initGUI_3031();
23.     }
24.
25.     private void initGraph_3031() {
26.
27.         graph_3031 = new LinkedHashMap<>();
28.
29.         String[] lokasi_3031 = {
30.             "Rektorat",
31.             "Perpustakaan",
32.             "FTI",
33.             "PKM",
34.             "Masjid",
35.             "Lab Komputer",
36.             "Kantin"
37.         };
38.
39.         for (String lokasi : lokasi_3031) {
40.             graph_3031.put(lokasi, new ArrayList<>());
41.         }
42.
43.         addEdge_3031("Rektorat", "Perpustakaan");
44.         addEdge_3031("Rektorat", "PKM");
45.         addEdge_3031("Rektorat", "Masjid");
46.
47.         addEdge_3031("Perpustakaan", "FTI");
48.
49.         addEdge_3031("FTI", "Lab Komputer");
```

```

50.
51.     addEdge_3031("PKM", "Lab Komputer");
52.
53.     addEdge_3031("Masjid", "Kantin");
54.
55.     addEdge_3031("Lab Komputer", "Kantin");
56. }
57.
58. private void addEdge_3031(String a, String b) {
59.     graph_3031.get(a).add(b);
60.     graph_3031.get(b).add(a);
61. }
62.
63. private void initGUI_3031() {
64.
65.     setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
66.     setSize(900, 650);
67.     setLocationRelativeTo(null);
68.     setDefaultCloseOperation(EXIT_ON_CLOSE);
69.     setLayout(new BorderLayout());
70.
71.     JPanel panelAtas_3031 = new JPanel(new
FlowLayout(FlowLayout.LEFT, 15, 10));
72.
73.     panelAtas_3031.add(new JLabel("Lokasi Awal :"));
74.
75.     cbAwal_3031 = new
JComboBox<>(graph_3031.keySet().toArray(new String[0]));
76.     panelAtas_3031.add(cbAwal_3031);
77.
78.     panelAtas_3031.add(new JLabel("Lokasi Tujuan :"));
79.
80.     cbTujuan_3031 = new
JComboBox<>(graph_3031.keySet().toArray(new String[0]));
81.     cbTujuan_3031.setSelectedIndex(5);
82.     panelAtas_3031.add(cbTujuan_3031);
83.
84.     JButton btnBFS_3031 = new JButton("[ BFS ]");
85.     JButton btnDFS_3031 = new JButton("[ DFS ]");
86.     JButton btnReset_3031 = new JButton("[ RESET ]");
87.
88.     panelAtas_3031.add(btnBFS_3031);
89.     panelAtas_3031.add(btnDFS_3031);
90.     panelAtas_3031.add(btnReset_3031);
91.
92.     add(panelAtas_3031, BorderLayout.NORTH);
93.
94.     graphArea_3031 = new JTextArea();
95.     graphArea_3031.setEditable(false);
96.     graphArea_3031.setFont(new Font("Monospaced", Font.PLAIN,
18));
97.
98.     graphArea_3031.setText(
99.         "\n" +

```

```

100.         "        Perpustakaan ----- FTI\n" +
101.         "        |                               |\n" +
102.         "        |                               |\n" +
103.         "        Rektorat ---- PKM ---- Lab
Komputer\n" +
104.         "        |                               |\n" +
105.         "        |                               |\n" +
106.         "        Masjid ----- Labor AI\n"
107.     );
108.
109.     JPanel panelGraph_3031 = new JPanel(new
BorderLayout());
110.     panelGraph_3031.setBorder(
111.         BorderFactory.createTitledBorder("VISUALISASI GRAPH")
112.     );
113.
114.     panelGraph_3031.add(graphArea_3031);
115.
116.     add(panelGraph_3031, BorderLayout.CENTER);
117.
118.     hasilArea_3031 = new JTextArea(8, 50);
119.     hasilArea_3031.setEditable(false);
120.     hasilArea_3031.setFont(new Font("Monospaced",
Font.PLAIN, 16));
121.
122.     hasilArea_3031.setText(
123.         "Hasil Pencarian :\n" +
124.         "Jalur :\n" +
125.         "Node Dikunjungi :\n" +
126.         "Jumlah Node Dikunjungi : 0"
127.     );
128.
129.     JPanel panelBawah_3031 = new JPanel(new
BorderLayout());
130.     panelBawah_3031.setBorder(
131.         BorderFactory.createTitledBorder("HASIL")
132.     );
133.
134.     panelBawah_3031.add(new
JScrollPane(hasilArea_3031));
135.
136.     add(panelBawah_3031, BorderLayout.SOUTH);
137.
138.     btnBFS_3031.addActionListener(e -> {
139.         BFS_3031();
140.         tampilkanHasil_3031("BFS");
141.     });
142.
143.     btnDFS_3031.addActionListener(e -> {
144.         DFS_3031();
145.         tampilkanHasil_3031("DFS");
146.     });
147.

```

```

148.         btnReset_3031.addActionListener(e -> {
149.             visited_3031.clear();
150.             path_3031.clear();
151.
152.             hasilArea_3031.setText(
153.                 "Hasil Pencarian :\n" +
154.                 "Jalur :\n" +
155.                 "Node Dikunjungi :\n" +
156.                 "Jumlah Node Dikunjungi : 0"
157.             );
158.         });
159.
160.         setVisible(true);
161.     }
162.
163.     private void BFS_3031() {
164.
165.         visited_3031.clear();
166.         path_3031.clear();
167.
168.         String start = (String)
169.             cbAwal_3031.getSelectedItem();
170.         String goal = (String)
171.             cbTujuan_3031.getSelectedItem();
172.
173.         Queue<java.util.List<String>> queue = new
174.             LinkedList<>();
175.         Set<String> seen = new HashSet<>();
176.
177.         java.util.List<String> firstPath = new
178.             ArrayList<>();
179.         firstPath.add(start);
180.
181.         queue.add(firstPath);
182.         seen.add(start);
183.
184.         while (!queue.isEmpty()) {
185.
186.             java.util.List<String> currentPath =
187.                 queue.poll();
188.             String current =
189.                 currentPath.get(currentPath.size() - 1);
190.
191.             visited_3031.add(current);
192.
193.             if (current.equals(goal)) {
194.                 path_3031 = currentPath;
195.                 return;
196.             }
197.
198.             for (String next : graph_3031.get(current)) {
199.                 if (!seen.contains(next)) {

```

```

196.                seen.add(next);
197.
198.                java.util.List<String> newPath =
199.                    new ArrayList<>(currentPath);
200.
201.                newPath.add(next);
202.
203.                queue.add(newPath);
204.            }
205.        }
206.    }
207.}
208.
209.    private void DFS_3031() {
210.
211.        visited_3031.clear();
212.        path_3031.clear();
213.
214.        String start = (String)
215.            cbAwal_3031.getSelectedItemAt();
216.        String goal = (String)
217.            cbTujuan_3031.getSelectedItemAt();
218.
219.        Stack<java.util.List<String>> stack = new
220.            Stack<>();
221.
222.        java.util.List<String> awal = new ArrayList<>();
223.        awal.add(start);
224.
225.        stack.push(awal);
226.
227.        while (!stack.isEmpty()) {
228.
229.            java.util.List<String> currentPath =
230.                stack.pop();
231.
232.            String current =
233.                currentPath.get(currentPath.size() -
234.                    1);
235.
236.            if (visited_3031.contains(current))
237.                continue;
238.
239.            visited_3031.add(current);
240.
241.            if (current.equals(goal)) {
242.                path_3031 = currentPath;
243.                return;
244.            }
245.
246.            java.util.List<String> neighbors =
247.                new
248.                ArrayList<>(graph_3031.get(current));
249.

```

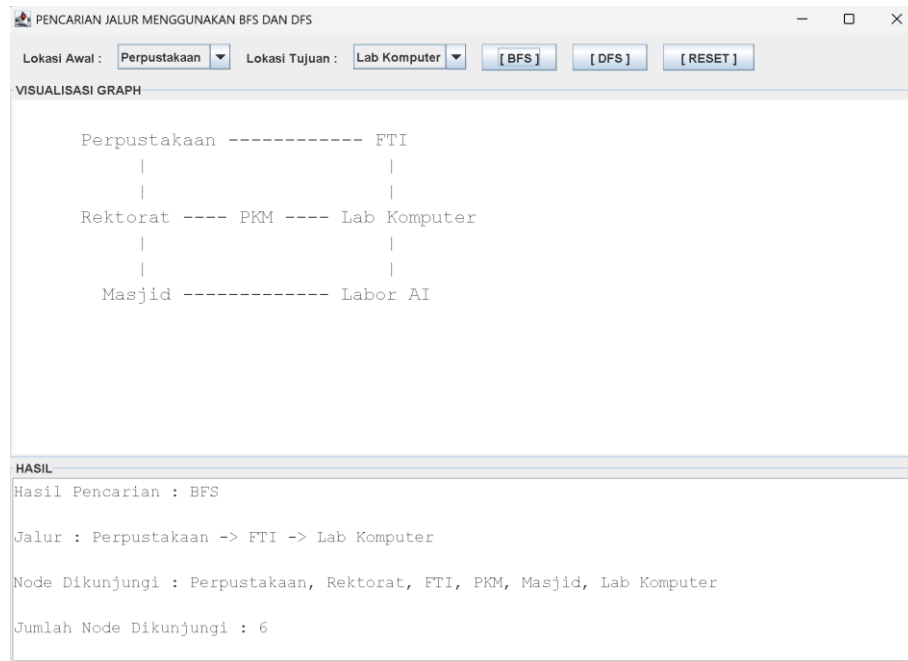
```

244.         Collections.reverse(neighbors);
245.
246.         for (String next : neighbors) {
247.
248.             if (!visited_3031.contains(next)) {
249.
250.                 java.util.List<String> newPath =
251.                     new ArrayList<>(currentPath);
252.
253.                 newPath.add(next);
254.
255.                 stack.push(newPath);
256.             }
257.         }
258.     }
259. }
260.
261. private void tampilkanHasil_3031(String metode) {
262.
263.     hasilArea_3031.setText(
264.         "Hasil Pencarian : " + metode + "\n\n" +
265.         "Jalur : " +
266.         String.join(" -> ", path_3031) + "\n\n" +
267.         "Node Dikunjungi : " +
268.         String.join(", ", visited_3031) + "\n\n" +
269.         "Jumlah Node Dikunjungi : " +
270.         visited_3031.size()
271.     );
272. }
273.
274. public static void main(String[] args) {
275.     new PetaKampus_2511533031();
276. }
277. }

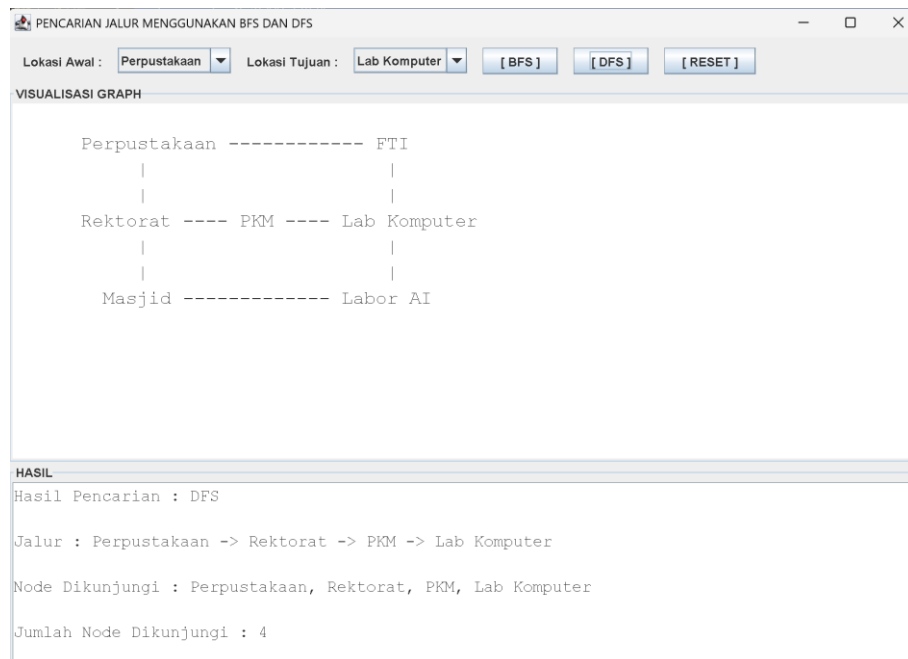
```

B. Output

1. BFS



2. DFS



C. Deskripsi Peta

Pada tugas ini dibuat sebuah peta sederhana lingkungan kampus yang direpresentasikan dalam bentuk graf tak berarah (undirected graph). Setiap

lokasi pada kampus direpresentasikan sebagai node, sedangkan jalur yang menghubungkan antar lokasi direpresentasikan sebagai edge. Program dibuat menggunakan bahasa pemrograman Java dan menerapkan algoritma Breadth First Search (BFS) serta Depth First Search (DFS) untuk mencari jalur dari lokasi awal menuju lokasi tujuan.

Lokasi yang digunakan dalam peta kampus terdiri dari Rektorat, Perpustakaan, FTI, PKM, Masjid, Lab Komputer, dan Kantin. Pengguna dapat memilih lokasi awal dan lokasi tujuan melalui antarmuka program, kemudian menjalankan pencarian menggunakan BFS atau DFS.

D. Jumlah Node dan Edge

Graf yang digunakan pada program memiliki 7 node dan 8 edge.

Node:

1. Rektorat
2. Perpustakaan
3. FTI
4. PKM
5. Masjid
6. Lab Komputer
7. Labor AI

Edge

1. Rektorat – Perpustakaan
2. Rektorat – PKM
3. Rektorat – Masjid
4. Perpustakaan – FTI
5. FTI – Lab Komputer
6. PKM – Lab Komputer
7. Masjid – Labor AI
8. Lab Komputer – Labor AI

E. Kesimpulan Perbandingan BFS dan DFS

Berdasarkan hasil pengujian, algoritma BFS dan DFS sama-sama berhasil menemukan jalur dari Rektorat menuju Lab Komputer. BFS menghasilkan jalur Rektorat → PKM → Lab Komputer yang merupakan jalur terpendek dengan dua edge. Sementara itu, DFS menghasilkan jalur Rektorat → Perpustakaan → FTI → Lab Komputer dengan tiga edge karena algoritma DFS menelusuri cabang pertama terlebih dahulu hingga mencapai tujuan.

Dapat disimpulkan bahwa BFS lebih efektif untuk mencari jalur terpendek pada graf, sedangkan DFS lebih cocok digunakan untuk melakukan penelusuran secara mendalam terhadap suatu cabang graf. Pemilihan algoritma dapat disesuaikan dengan kebutuhan pencarian yang dilakukan.